

Balloon Popping Challenge

Monthly Technical Workshop

Prepared by ZuoRen Chen

Organizer: Taiwan Space Agency (TASA)

Co-organizer: Advanced Rocket Research Center (ARRC)

Supervisor: National Science and Technology Council (NSTC)

Contact: Education Office, TASA: chchiang@tasa.org.tw

Codebase maintainers: [chichunwang](#), [thc1006](#), [William-Mou](#)

May 30th, 2026





Outline

1 Introduction

2 Code base

3 Preliminary Competition Rules

4 Upcoming Updates

5 Q&A

1

Introduction

Introduction

Inspiration

[Join AIAA](#)[Log In](#)[Donate](#)[EVENTS & LEARNING](#)[GET INVOLVED](#)[PUBLICATIONS](#)[NEWS](#)[ADVOCACY](#)[CAREERS](#)[MEMBERSHIP](#)

2026 Capture the Satellite Challenge

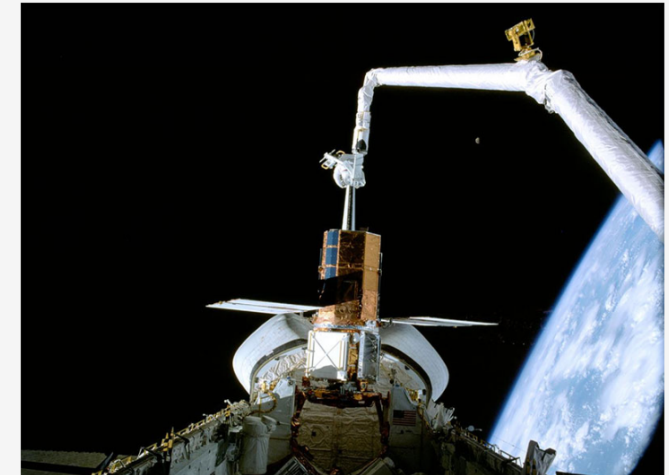
AIAA SciTech Forum and Exposition / Program

An Autonomy Design Competition for Non-Cooperative Space Operations in Kerbal Space Program

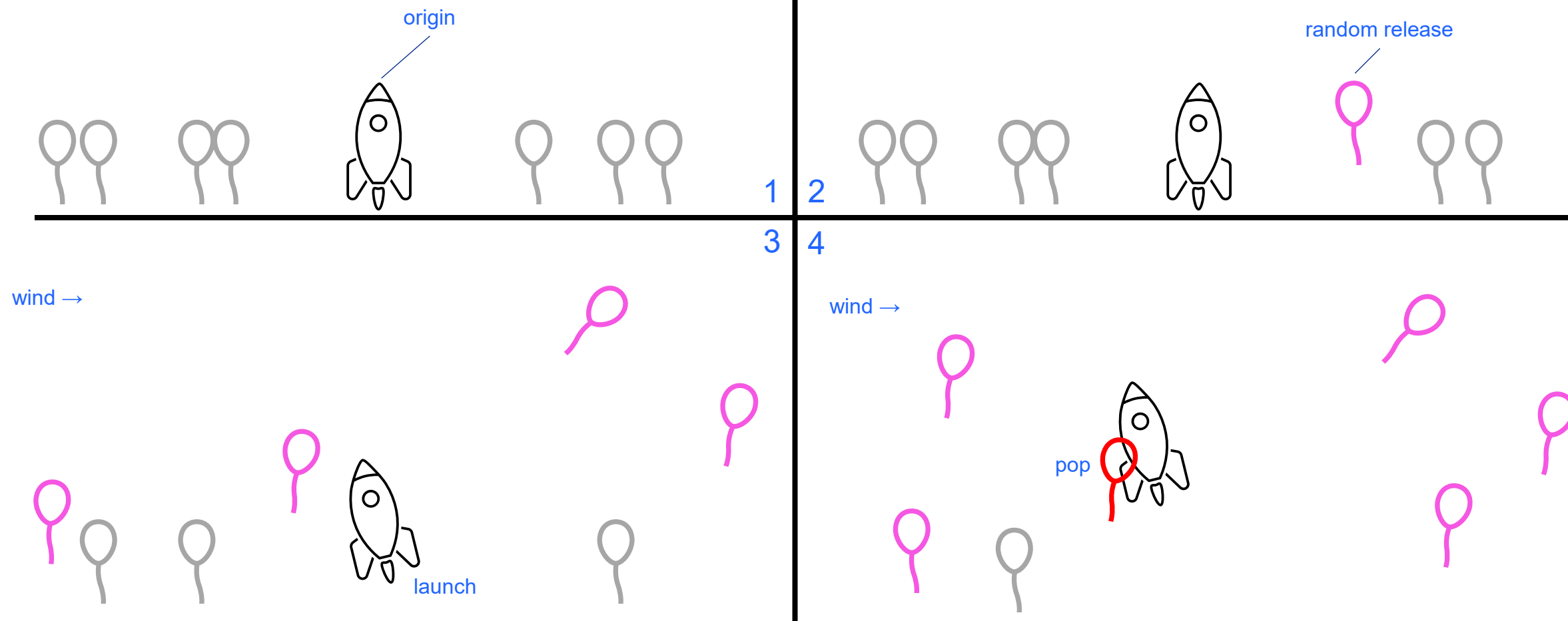
[Sign Up Now](#)

Join us for the third annual software design competition where participants develop autonomous agents for maneuvering satellites engaged in non-cooperative space operations.

- The software design challenge is based upon MIT Lincoln Laboratory's Kerbal Space Program Differential Games (KSPDG) **open-source library**.
- Participants in the challenge will develop agents and algorithms for autonomous control of spacecraft simulated using Kerbal Space Program.
- The agents will seek to solve several challenging problems related to non-cooperative space operations, such as pursuing an evasive satellite and conducting multi-satellite proximity operations.
- Participants are encouraged to develop agents using a range of technologies, such as reinforcement learning, optimal control, and game-theoretic techniques.
- Participants' agents will be evaluated on a range of metrics, including mission completion time, fuel consumption, and relative distances achieved.
- The participant pool will undergo a series of elimination events and teams with the best-performing agents will be invited to present their agents via a live demonstration during AIAA SciTech Forum in January 2026.
- Monthly meetings will be held in the lead up to AIAA SciTech Forum to allow technical exchanges between participants, MIT Lincoln Laboratory developers, and AIAA SciTech Forum organizers.



Introduction



Introduction

Overview

Objective

This competition aims to cultivate next-generation talent in rocket Guidance, Navigation, and Control (GNC) by engaging participants in the development of autonomous flight control algorithms.

Participants are required to develop Python-based software to autonomously control a simulated rocket. Within a physics-based simulation environment, the rocket must navigate and pop balloons released dynamically to maximize the number of pops under uncertain conditions.

Eligibility

- The competition is open to international participants, including high school students, university students, researchers, and professionals. Participants with an interest in rocket Guidance, Navigation, and Control (GNC) and relevant programming experience are encouraged to apply.
- Participants must register as a team consisting of one (1) to five (5) members. Team members may come from different institutions or backgrounds.
- Each individual may participate in only one team. Multiple registrations are not permitted.
- This competition is conducted in an international format. All official rules, documentation, and software interfaces will be provided in English.

Awards and Resources

All finalist teams will receive a Certificate of Participation (electronic copy). Awards will be granted based on final rankings as follows:

- 1st Place: NTD 50,000 per team and an award certificate
- 2nd Place: NTD 30,000 per team and an award certificate
- 3rd Place: NTD 10,000 per team and an award certificate
- Finalist teams will be invited to attend selected sessions of TASTI 2026 free of charge.

Introduction

Timeline

[Registration link](#)



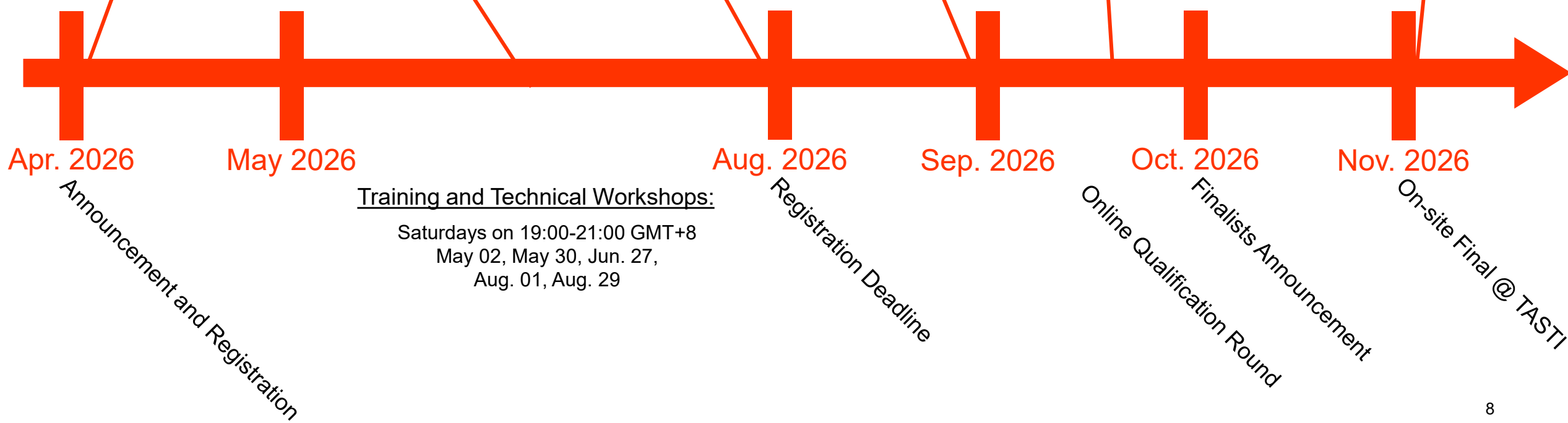
- Monthly technical workshops
- **Software updates**
- Rule refinements

Top 10 teams on leaderboard
will enter qualification round

- 3-5 teams in the final
- Technical presentation on agents
- Live demo

Finalized software and rule

Qualification round rules
to be announced



Introduction

Timeline

	Event	Time	Link
	1 st Workshop	May 02 (Sat.) GMT+8 19:00-21:00	Recording
	2 nd Workshop	May 30 (Sat.) GMT+8 19:00-21:00	Link
	3 rd Workshop	Jun. 27 (Sat.) GMT+8 19:00-21:00	Link
	4 th Workshop	Aug. 01 (Sat.) GMT+8 19:00-21:00	Link
	5 th Workshop	Aug. 29 (Sat.) GMT+8 19:00-21:00	Link
	Online Qualification	Sep. TBD	
	On-site Final @ TASTI	Nov. TBD (8/9/10/11)	

Purpose of workshops:

1. Announce rule and code base updates
2. Answer technical / admin questions from participants
3. Technical exchange between participants

Updated

Introduction

Reading Materials

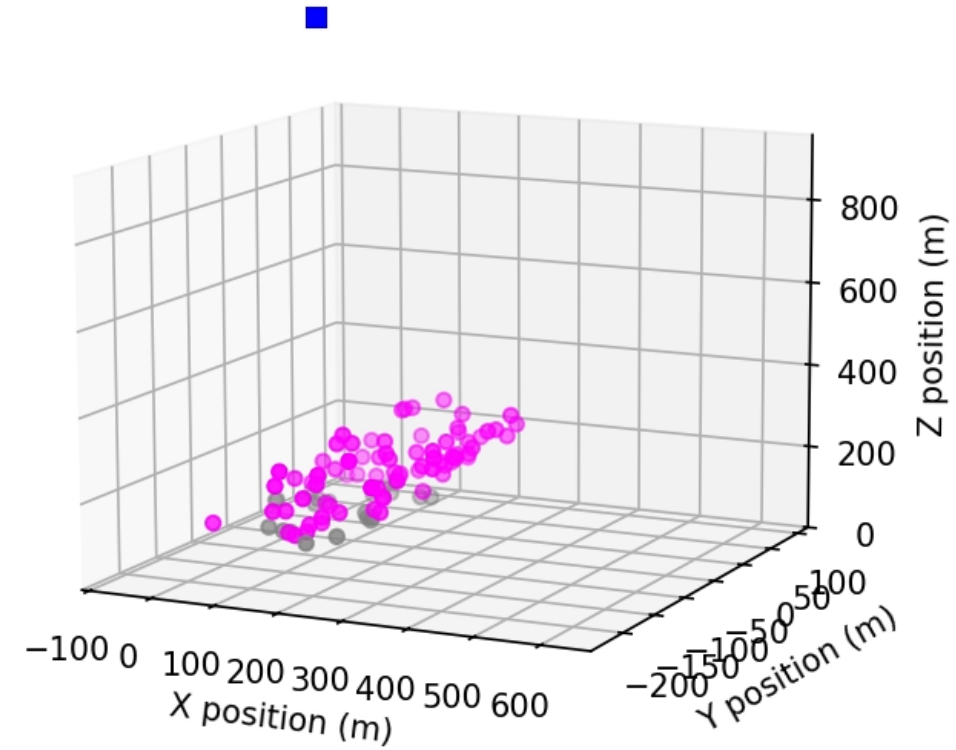
Welcome for suggestions to this list!

1. Woodman, O. J. (2007). *An introduction to inertial navigation* (Research report696). University of Cambridge.
2. Grewal, M.S., Andrews, A.P. and Bartone, C.G. (2020). *Global Navigation Satellite Systems, inertial navigation, and Integration*. Hoboken, NJ: Wiley.
3. Thrun, S., Burgard, W.,, Fox, D. (2005). *Probabilistic robotics*. Cambridge, Mass.: MIT Press. ISBN: 0262201623 9780262201629
4. Åström, K. J., Wittenmark, B. (1984). *Computer Controlled Systems: Theory and Design*. Prentice Hall.
5. Russell, S. and Norvig, P. (2010) *Artificial Intelligence: A Modern Approach*. 3rd Edition, Prentice-Hall, Upper Saddle River.

2

Code Base

Time: 39.40 sec
Reward: 0



Code Base

Overview

Platform:

Participants will develop the software from [BalloonPoppingChallenge](#). This is based on [ActiveRocketPy](#), a fork of open-source software RocketPy.

Scenario:

Participants are required to develop algorithms to control a rocket equipped with a throttleable propulsion system and thrust vector control (TVC). The rocket will be launched from the center of a simulation field of hundreds of meters. Approximately 100 (final value to be specified) target balloons will be randomly distributed within the field and released from the ground at randomized time intervals and sequences.

The simulation environment will incorporate uncertainties and disturbances, including but not limited to wind effects and sensor noise. The launch timing and control strategy must be autonomously determined by the participant's GNC algorithm. The objective is to maximize successful pops under dynamic and uncertain conditions.

(Final scenario parameters will be specified in the official competition release.)

Technical Scope:

A wide range of control methodologies is encouraged, including but not limited to:

- Classical control
- Modern control
- Optimal control
- Machine learning and reinforcement learning approaches

Keywords: GNC, autonomous flight control, optimization, path planning

[BalloonPoppingChallenge](#)

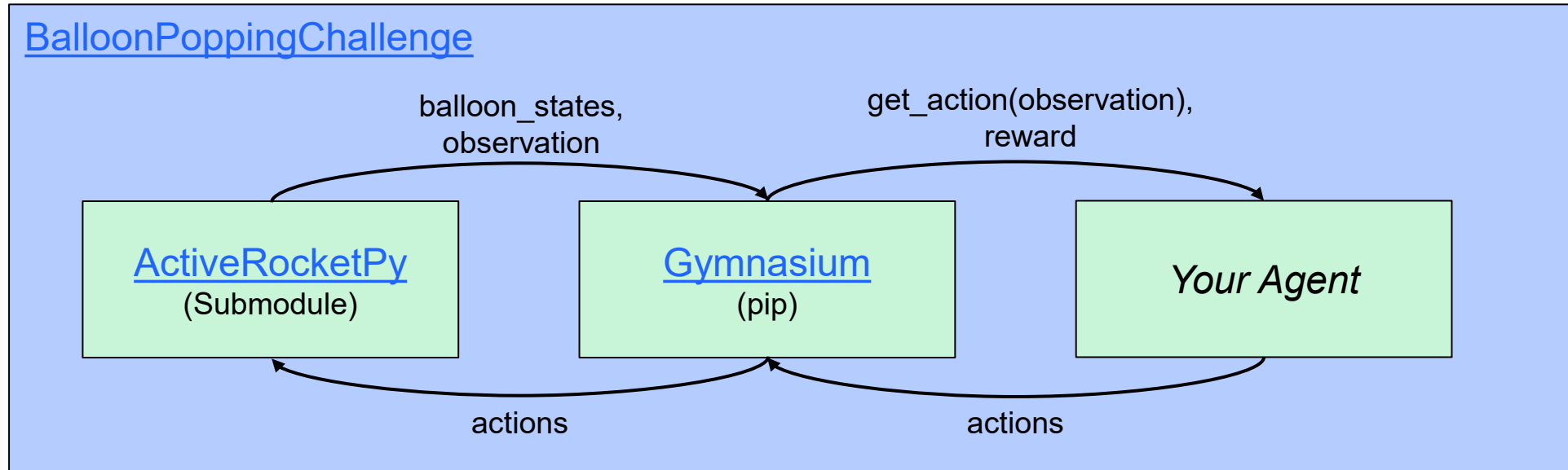


[ActiveRocketPy](#)



Code Base

Overview



Developed in Windows Python 3.14

Feel free to report [issues](#) and contribute by sending [pull requests](#)!

Code Base

Getting Started

Installation:

1. `git clone https://github.com/ARRC-Rocket/BalloonPoppingChallenge.git`
2. `cd BalloonPoppingChallenge`
3. `git submodule update --init` # Initialize the ActiveRocketPy submodule
4. `python -m venv .venv` # Create a virtual environment (optional but recommended)
5. `.venv\Scripts\activate` # On Windows
6. `# source .venv/bin/activate` # On Unix or MacOS
7. `python -m pip install -r requirements.txt`

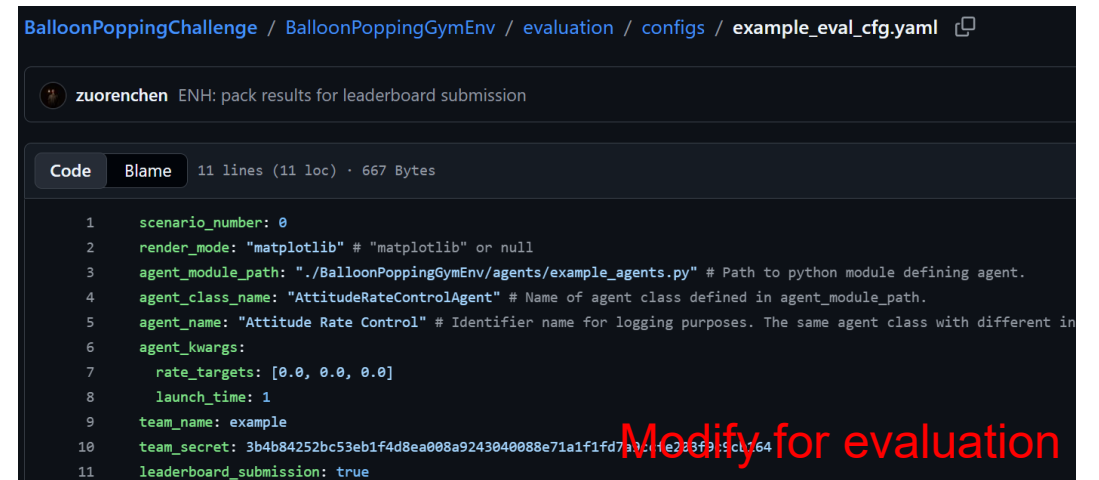
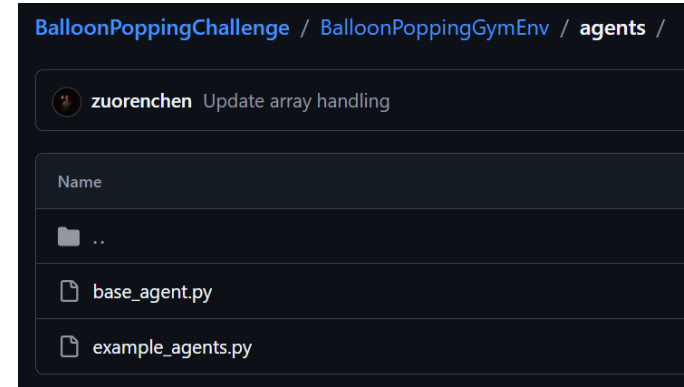
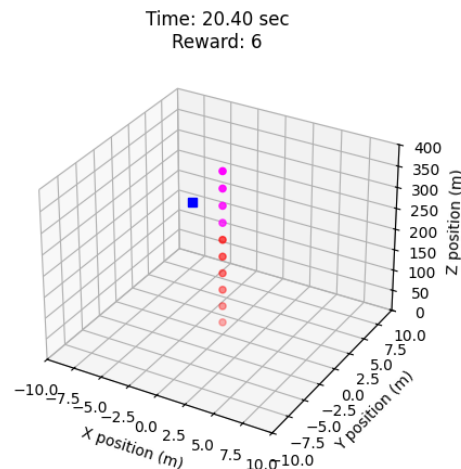
Update from the repository:

1. `cd BalloonPoppingChallenge`
2. `git pull origin main`
3. `git submodule update --remote --merge`

Code Base

Getting Started - Evaluate Agent

- Develop the agent in [/agents folder](#) by inheriting from [BaseAgent](#) and implementing the `get_action()` method.
- Update the evaluation configuration file in [/evaluation/configs folder](#) to specify the scenario parameters and the agent to be evaluated.
- Run
 - `cd BalloonPoppingChallenge`
 - `python .\BalloonPoppingGymEnv\evaluation\evaluate.py`
`.\BalloonPoppingGymEnv\evaluation\configs\example_eval_cfg.yaml`
- You should see a rocket popping static balloons in the sky:



Check [README.md in BalloonPoppingChallenge](#) for more details

Code Base

Scenario and Parameters

BalloonPoppingChallenge / BalloonPoppingGymEnv / evaluation / configs / example_eval_cfg.yaml

zuorenchen Update to add throttle control

Modify for evaluation

Code Blame 9 lines (9 loc) · 557 Bytes

```

1 scenario_number: 0
2 render_mode: "matplotlib" # "matplotlib" or null
3 agent_module_path: "../BalloonPoppingGymEnv/agents/example_agents.py" # Path to python module defining agent.
4 agent_class_name: "AttitudeRateControlAgent" # Name of agent class defined in agent_module_path.
5 agent_name: "Attitude Rate Control" # Identifier name for logging purposes. The same agent class with different
6 agent_kwargs:
7   rate_targets: [0.0, 0.0, 0.0]
8   launch_time: 1
9   arg3: value3

```

BalloonPoppingChallenge / BalloonPoppingGymEnv / envs / scenario_parameters / scenario_0_parameters.yaml

zuorenchen Update readme

Shouldn't modify these parameters for scenario evaluation

Code Blame 188 lines (184 loc) · 5.64 KB

```

1 # Scenario metadata
2 scenario:
3   number: 0
4   random_seed: 0 # Use null to enable random seeding
5
6 # Launch site and atmosphere setup
7 environment:
8   date: [2025, 7, 26, 0]
9   latitude: 22.1749259 # (deg) latitude of the launch origin
10  longitude: 120.8922531 # (deg) longitude of the launch origin
11  elevation: 20.0 # (m) elevation above sea level
12  atmosphere_data_filename: null # filename of the atmosphere data in the envs/data folder, or null to use standard atmosphere
13
14 # Simulation timing configuration
15 simulation:
16   time_step: 0.01 # (sec) time step for Gym environment, ActiveRocketPy controller, and balloon simulation
17   max_time: 100 # (sec) maximum simulation time of the scenario
18
19 # Balloon model and uncertainty parameters
20 balloon:
21   release_interval: 1 # no actual effect in scenario 0
22   num: 10 # number of balloons
23   radius: 1.5 # (m) radius of the balloon
24   mass: 0.8 # (kg) mass of the balloon
25   inertia: [1, 1, 1] # (kg*m^2) inertia of the balloon
26   aero_coefficients:

```

BalloonPoppingChallenge / BalloonPoppingGymEnv / envs / scenario_parameters / scenario_0_given_parameters.yaml

zuorenchen Add tank_position to given parameters

Code Blame 84 lines (84 loc) · 1.57 KB

```

1 # Parameters exposed to agents for scenario 0
2 environment:
3   - date
4   - latitude
5   - longitude
6   - elevation
7 simulation:
8   - time_step
9   - max_time
10 balloon:
11   - release_interval
12   - num
13   - radius
14   - mass
15 rocket:
16   tank:
17     - liquid
18     - liquid_density
19     - gas
20     - gas_density
21     - radius
22     - height
23     - flux_time
24     - initial_liquid_mass
25     - initial_gas_mass
26     - liquid_mass_flow_rate_out
27     - tank_position

```

Shouldn't modify the scenario parameters passed to agent when evaluating

Code Base

Example Agent

BalloonPoppingChallenge / BalloonPoppingGymEnv / evaluation / configs / example_eval_cfg.yaml

zuorenchen Update to add throttle control

Code Blame 9 lines (9 loc) · 557 Bytes

```

1 scenario_number: 0
2 render_mode: "matplotlib" # "matplotlib" or null
3 agent_module_path: "../BalloonPoppingGymEnv/agents/example_agents.py" # Path to python module defining agent.
4 agent_class_name: "AttitudeRateControlAgent" # Name of agent class defined in agent_module_path.
5 agent_name: "Attitude Rate Control" # Identifier name for logging purposes. The same agent class with different
6 agent_kwargs:
7   rate_targets: [0.0, 0.0, 0.0]
8   launch_time: 1
9   arg3: value3

```

BalloonPoppingChallenge / BalloonPoppingGymEnv / envs / scenario_0_parameters.yaml

zuorenchen Add tank_position to given parameters

Code Blame 84 lines (84 loc) · 1.57 KB

```

1 # Parameters exposed to agents for scenario 0
2 environment:
3   - date
4   - latitude
5   - longitude
6   - elevation
7 simulation:
8   - time_step
9   - max_time
10 balloon:
11   - release_interval
12   - num
13   - radius
14   - mass
15 rocket:
16   tank:
17     - liquid
18     - liquid_density
19     - gas
20     - gas_density
21     - radius
22     - height
23     - flux_time
24     - initial_liquid_mass
25     - initial_gas_mass
26     - liquid_mass_flow_rate_out
27     - tank_position

```

BalloonPoppingChallenge / BalloonPoppingGymEnv / agents / example_agents.py

Code Blame 231 lines (193 loc) · 8.29 KB

Develop your agents here

```

48
49 class AttitudeRateControlAgent(BaseAgent):
50     """An agent that controls attitude rates using a PID controller and launches at t=1s"""
51
52     def __init__(self, *args, **kwargs):
53         super().__init__(*args)
54         # Initialize an array to store attitude rate errors for PID control angular rates of (x, y, z) axis
55         # Shape: (3, n_timesteps) where each row tracks one axis over time
56         self.rate_errors = np.zeros((3, 1))
57         # Default target are 0 rad/s
58         self.rate_targets = np.array(kwargs.get("rate_targets", [0.0, 0.0, 0.0]))
59         self.launch_time = kwargs.get("launch_time", 1.0)
60
61     def get_action(self, observation):
62         """compute agent's action given observation
63
64         This function is necessary to define as it overrides
65         an abstract method
66         """
67         sensor_frequency = self.given_parameters["rocket"]["sensors"]["sampling_rate"]
68
69         if observation["simulation_time"] >= self.launch_time:
70             launch = True
71         else:
72             launch = False
73

```

Code Base

Example Agent

BalloonPoppingChallenge / BalloonPoppingGymEnv / agents / example_agents.py

Code Blame 231 lines (193 loc) · 8.29 KB

```
48
49 class AttitudeRateControlAgent(BaseAgent):
50     """An agent that controls attitude rates using a PID controller and launches at t=1s"""
51
52     def __init__(self, *args, **kwargs):
53         super().__init__(*args)
54         # Initialize an array to store attitude rate errors for PID control angular rates of (x, y, z) axis
55         # Shape: (3, n_timesteps) where each row tracks one axis over time
56         self.rate_errors = np.zeros((3, 1))
57         # Default target are 0 rad/s
58         self.rate_targets = np.array(kwargs.get("rate_targets", [0.0, 0.0, 0.0]))
59         self.launch_time = kwargs.get("launch_time", 1.0)
60
61     def get_action(self, observation):
62         """compute agent's action given observation
63
64         This function is necessary to define as it overrides
65         an abstract method
66         """
67         sensor_frequency = self.given_parameters["rocket"]["sensors"]["sampling_rate"]
68
69         if observation["simulation_time"] >= self.launch_time:
70             launch = True
71         else:
72             launch = False
```

User parameters in cfg.yaml are passed to agent at __init__

Given parameters in each scenario is saved in self.given_parameters

Agent should determine time, inclination, and heading of launch

```
73
74 if not np.isnan(observation["rocket_sensors"][:3]).any():
75     KP = np.array([100.0, 100.0, 100.0])
76     KI = np.array([0.0, 0.0, 5.0])
77     KD = np.array([0.0, 0.0, 0.0])
78
79     self.rate_errors = np.append(
80         self.rate_errors,
81         (self.rate_targets - observation["rocket_sensors"][:3]).reshape(-1, 1),
82         axis=1,
83     )
84     roll_rate_error_integrals = (
85         np.sum(self.rate_errors, axis=1) / sensor_frequency
86     )
87     roll_rate_error_derivatives = (
88         (self.rate_errors[:, -1] - self.rate_errors[:, -2]) * sensor_frequency
89         if self.rate_errors.shape[1] > 1
90         else np.array([0.0, 0.0, 0.0])
91     )
92     torque_cmd = (
93         KP * self.rate_errors[:, -1]
94         + KI * roll_rate_error_integrals[:3]
95         + KD * roll_rate_error_derivatives[:3]
96     )
97 else:
98     torque_cmd = np.array([0.0, 0.0, 0.0])
99
100 return {
101     "launch": launch,
102     "launch_inclination_heading": np.array([90, 0]),
103     "tvc": np.array([torque_cmd[0], torque_cmd[1]]),
104     "roll": torque_cmd[2],
105     "throttle": 1.0,
106 }
```

Develop user GNC algorithm here.
Takes observations then return actions

Return actions

Code Base

Colab Example

BalloonPoppingChallenge Scenario Evaluation in Google Colab

This notebook runs scenario evaluations for the BalloonPoppingChallenge using a trained agent. It automates setup, dependency installation, and evaluation execution.

Section 0: Setup and Installation

Set up the Colab environment by installing required system dependencies and Python packages.

Restart runtime after pip install!

Restart runtime after pip install

```
In [ ]: %cd /content
        !git clone https://github.com/ARRC-Rocket/BalloonPoppingChallenge.git
        %cd BalloonPoppingChallenge
        !git submodule update --init # Initialize the ActiveRocketPy submodule
        !python -m pip install -r requirements.txt
```

Section 1: Import Required Libraries

Import necessary libraries for the evaluation.

```
In [ ]: import numpy as np

        from BalloonPoppingGymEnv.agents.base_agent import BaseAgent
        from BalloonPoppingGymEnv.evaluation.evaluate import evaluate_scenario
```

Section 2: Develop Agent

Develop your agent here in this notebook. You can also import your agent from a separate Python file if you prefer.

```
In [ ]: # from from BalloonPoppingGymEnv.agents.your_agent import YourAgent

        class NoActionAgent(BaseAgent):
            """An agent that does nothing"""

            def __init__(self, *args, **kwargs):
                super().__init__(*args)

                self.launch_time = kwargs.get("launch_time", 0.0)

            def get_action(self, observation):
                """compute agent's action given observation

                This function is necessary to define as it overrides
                an abstract method
                """

                if observation["simulation_time"] >= self.launch_time:
                    launch = True
                else:
                    launch = False

                return {
                    "launch": launch,
                    "launch_inclination_heading": np.array([90, 180]),
                    "tvc": np.array([0, 0]),
                    "roll": 0,
                    "throttle": 1,
                }
```

Section 3: Run Evaluation

Evaluate the agent.

```
In [ ]: scenario_number = 0

        render_mode = None # rendering not supported in notebook environment yet
        agent_name = "no_action_agent"
        agent_kwargs = {"launch_time": 1.0}

        # Load agent class dynamically from specified module path.
        evaluate_scenario(
            NoActionAgent,
            agent_kwargs=agent_kwargs,
            agent_name=agent_name,
            scenario_number=scenario_number,
            render_mode=render_mode,
        )
```

Code Base

Known limitations

1. The mass properties are pre-calculated before flight according to max flow rate and burn time. Throttle commands does not affect the change of the mass properties in-flight. It is equivalent to throttling the Isp of rocket engine while the flow rate remains constant. The engine is cut off when burn time is reached.
2. The aerodynamics model is only suitable for small angle of attack (AoA). Thus, AoA is saturated to a small angle in aerodynamic force calculation.

Code Base

Change Log

Fix/maintain:

- Fix json file read encoding issue (🔗 [Fix evaluate's encode problem #5](#))
- Make vpython an optional, lazily-imported render dependency (🔗 [Make vpython an optional, lazily-imported render dependency #15](#))
- Clean up dead code, redundant calls, and unused imports (🔗 [Clean up dead code, redundant calls, and unused imports #7](#))
- Move Monte Carlo output out of the package directory (🔗 [Move Monte Carlo output out of the package directory #22](#))
- Add GitHub Actions CI workflow ((🔗 [Add GitHub Actions CI workflow #23](#)))

Update:

- Update to return reward from each step only. Total count of popped balloon is returned in info. (👍 [Reward returned every step is the cumulative pop count, not the per-step delta #10](#)) (🔗 [ENH: Return reward of each step #12](#))
- Skip Monte Carlo balloon simulation for scenario 0 (🔗 [Skip Monte Carlo balloon simulation for scenario 0 #18](#))
- Render every balloon in the vpython renderer ((🔗 [Render every balloon in the vpython renderer #27](#)))

New:

- Add uv as the recommended environment setup path (🔗 [Add uv as the recommended environment setup path #20](#))
- Add standardized issue templates (🔗 [Add standardized issue templates #25](#))
- ENH: Add architecture for post sim graphics ((🔗 [ENH: Add architecture for post sim graphics #28](#)))
- ENH: Add leaderboard submission functions ((🔗 [ENH: Add leaderboard submission functions #31](#)))
- Add gust to balloon_world environment ((🔗 [Add gust to balloon_world environment #32](#)))

New

Code Base

Change Log - Gust

Time: 0.90 sec
Total Reward: 0

```

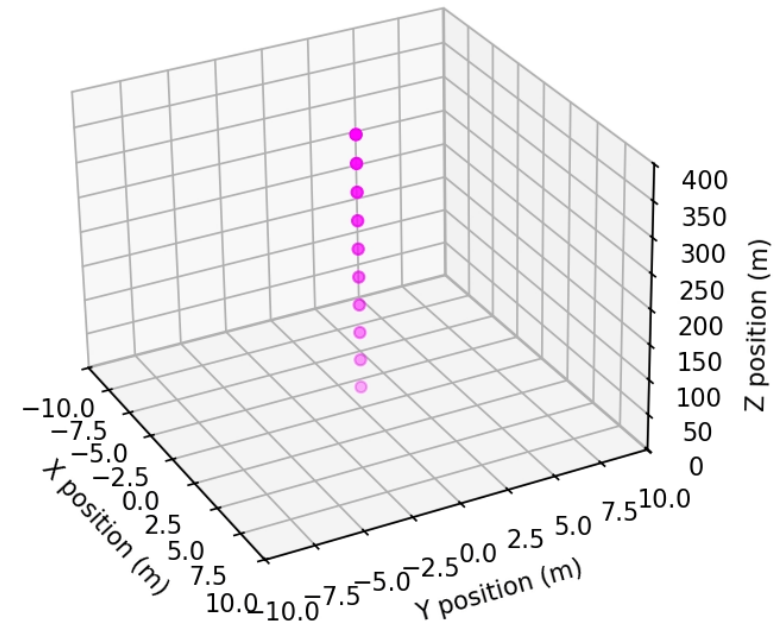
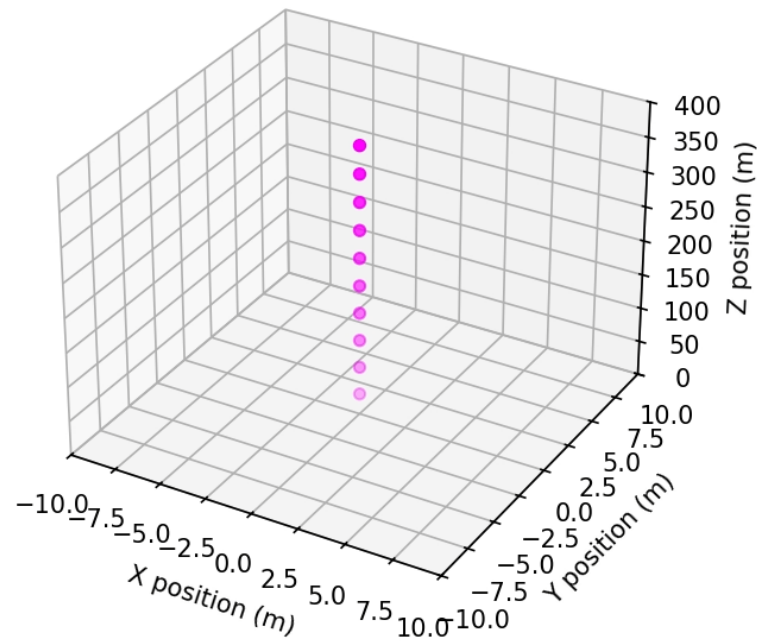
BalloonPoppingChallenge / BalloonPoppingGymEnv / envs / scenario_parameters / scenario_0_parameters.yaml
zuorenchen Refactor quasi-random gust model

Code Blame 113 lines (189 loc) · 5.88 KB

1 # Scenario metadata
2 scenario:
3   number: 0
4   random_seed: 0 # Use null to enable random seeding
5
6 # Launch site and atmosphere setup
7 environment:
8   date: [2025, 7, 26, 0]
9   latitude: 22.1749259 # (deg) latitude of the launch origin
10  longitude: 120.8922531 # (deg) longitude of the launch origin
11  elevation: 20.0 # (m) elevation above sea level
12  atmosphere_data_filename: null # filename of the atmosphere data in the envs/data folder, or null to use standard atmosphere
13
14  gust:
15    enable: false
16    altitude_spacing: 10.0 # (m) vertical spacing between gust samples
17    max_gust_speed: 3.0 # (m/s) maximum gust speed
18    gust_decay_height: 150.0 # (m) characteristic decay height for the gust speed with altitude

```

Time: 0.90 sec
Total Reward: 0



Code Base

[Change Log – vPython Render](#)

Balloon Popping Environment



New

3

Preliminary Competition Rules

Preliminary Competition Rules

Development

- **Honor code:** if you're doing something clever to the code base to improve agent performance, you might be cheating. When in doubt, ask organizer for clarification. -- **Cheaters will be disqualified**
- The participant will develop agents in /agents folder to control a rocket.
- The agent will be initialized with the given parameter of each scenario and user kwarg defined in cfg.yaml.
- At each time step, the agent should only take the observations provided by the environment to output control commands (e.g., launch, roll, throttle and TVC commands). The agent should not have access to any other information about the environment or the simulator.
- Other than the agent, all other components of the simulator are fixed and provided by the organizer. Participants are not allowed to modify any other part of the codebase for the evaluation.
- Evaluation results, including final reward and flight details, will be encrypted before uploading to leaderboard.
- Top 10 teams with highest score on leaderboard will enter qualification round.

Preliminary Competition Rules

Competition

- **Honor code:** if you're doing something clever to the code base to improve agent performance, you might be cheating. When in doubt, ask organizer for clarification. -- **Cheaters will be disqualified**
- Evaluation results, including final reward and flight details, will be encrypted before uploading to leaderboard.
 - **TBD:** scenario for leaderboard ranking
- Top 10 teams with highest score on leaderboard will enter qualification round.
- Participants of qualification and final rounds shall share the source code of agents with the organizer for examination.
- In qualification event, teams will evaluate their agents live during online meeting.
 - **TBD:** Scenario and parameters in qualification event will only be announced a week before the event.
 - **TBD:** Random seed will be picked right before each team evaluate their agent live during online meeting.
 - train / develop your agent to cover all variations

Preliminary Competition Rules

Notes

- Questions about the rules and software can be asked in the GitHub Issues. The organizer will answer them as soon as possible.
- Monthly workshops include updates and Q&A sessions.
- Code suggestions, contributions, and bug reports to the codebase are highly welcomed. Please submit a pull request or open an issue for discussion.

Preliminary Competition Rules

Scenarios

#	Name	 Actuator Response	 Sensor Noise	 Wind	 Number	 Release Interval (sec)	 Initial Position	 Position Observation	 Velocity Observation
#0	Hello World	Ideal	No	None	10	N/A	height = arange(10, 410, 40) + elevation	Static at initial position	no velocity
#1	Random Balloon	Ideal	No	Yes	100	Random	Random at ground	Full observation at current step	Full observation at current step
#2 (TBD)	Noisy Sensor	Ideal	Yes	Yes	100	Random	Random at ground	Full observation at current step	Full observation at current step
#3 (TBD)	Clumsy Actuator	LPF, random	Yes	Yes	100	Random	Random at ground	Full observation at current step	Full observation at current step
#4 (TBD)	Bad Weather	LPF, random	Yes	Yes, random magnitude	100	Random	Random at ground	Full observation at current step	Full observation at current step
#5 (TBD)	Sensor Drop off	LPF, random	Yes & drop-off	Yes, random magnitude	100	Random	Random at ground	Full observation at current step	Full observation at current step
#6 (TBD)	Find the Balloon	LPF, random	Yes & drop-off	Yes, random magnitude	100	Random	Random at ground	Partial observation at current step	Partial observation at current step
#7 (TBD)	Balloon Recovery								

Preliminary Competition Rules

Leaderboard

- Website link: <https://balloonpoppingchallenge.arrocket.org/>
- Evaluation for submission:
 - Update eval_cfg.yaml
 - Run evaluation.py
- team_secrets will be sent through emails next week

BalloonPoppingChallenge / BalloonPoppingGymEnv / evaluation / configs / example_eval_cfg.yaml

zuorenchen ENH: pack results for leaderboard submission

Code Blame 11 lines (11 loc) · 667 Bytes

```
1 scenario_number: 0
2 render_mode: "matplotlib" # "matplotlib" or null
3 agent_module_path: "./BalloonPoppingGymEnv/agents/example_agents.py" # Path to python module defining agent.
4 agent_class_name: "AttitudeRateControlAgent" # Name of agent class defined in agent_module_path.
5 agent_name: "Attitude Rate Control" # Identifier name for logging purposes. The same agent class with different in
6 agent_kwargs:
7   rate_targets: [0.0, 0.0, 0.0]
8   launch_time: 1
9 team_name: example
10 team_secret: 3b4b84252bc53eb1f4d8ea008a9243040088e71a1f1fd7a9ccfe203f9c9cb164
11 leaderboard_submission: true
```

Balloon Popping Leaderboard

Upload result .pk1 to view scores and trajectory replay

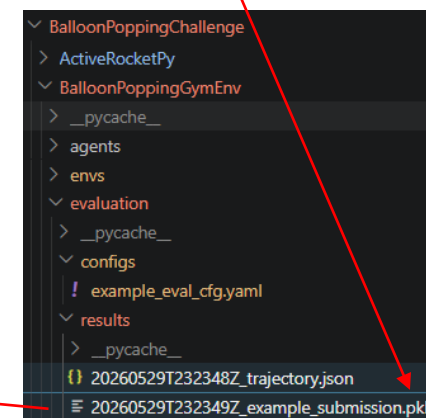
Submit Result

Choose File No file chosen

Upload

Leaderboard

#	Team	Agent	Score	Scenario	Submitted At	
1	example	Attitude Rate Control	10	0	5/30/2026, 12:42:28 AM	Replay



New

Preliminary Competition Rules

Leaderboard

Trajectory Replay

example · Attitude Rate Control · 成績 10

Play

Speed

1×

Time

6.34s

中文

Right-drag pan · Mouse wheel zoom · Left-drag orbit · Space play/pause · Arrow keys pan

Time & Scenario

Sim time6.34 s

Frame533 / 5911

PlaybackPaused · 1×

Scenario0

Step / balloon radius0.01 s · 1.5 m

Score10

Rocket (sim coords · z = altitude)

Position (x, y, z)(-0.0, 0.0, 53.3)

Altitude z53.3 m

Velocity (vx, vy, vz)(-0.0, 0.0, 12.7)

|v|12.7 m/s

Flight path angle90.0°

Attitude quaternion (e0-e3)(1.000, -0.000, -0.000, -0.000)

Angular rate ω (rad/s)(-0.000, -0.000, 0.000)

Balloons

Balloon status

Nearest target #1

Per-balloon

Ground 0 · Released 9 · Popped 1

Δ3D 16.7 m · horiz 0.0 m · Δz 16.7 m · at (0.0, 0.0, 70.0)

#0Popped · #1Released · #2Released · #3Released · #4Released · #5Released · #6Released · #7Released · #8Released · #9Released

Camera

Position Three (x, y1, z)(343.1, 487.8, 558.3)

Position sim (x, y, z1)(343.1, 558.3, 487.8)

Look-at Three(-162.2, -94.4, -337.3)

Look-at sim(-162.2, -337.3, -94.4)

View direction (unit)(-0.428, -0.493, -0.758)

Distance to look-at1181.7 m

New

4

Upcoming Updates

Upcoming Updates

- Evaluation time limit (wall time)
- Actuator Dynamics
- New scenarios

Update

5

Q&A

Q & A

- Questions to organizers
 - *Feel free to ask in Mandarin Chinese, I can translate to English for you.*
 - *(For other languages, I'll ask Google to translate, sorry 😅)*
- Technical exchange

Q & A

From previous meetings

First Workshop:

Q1: Does changing ISP of throttling affect TVC?

A1: The TVC is controlled by giving gimbal angle commands. The ISP reduced trust will make pitch/yaw torque from TVC less effective but not affected.

Q2: Are the controls applied at the same rate as simulation step? (Is the command instantaneous?)

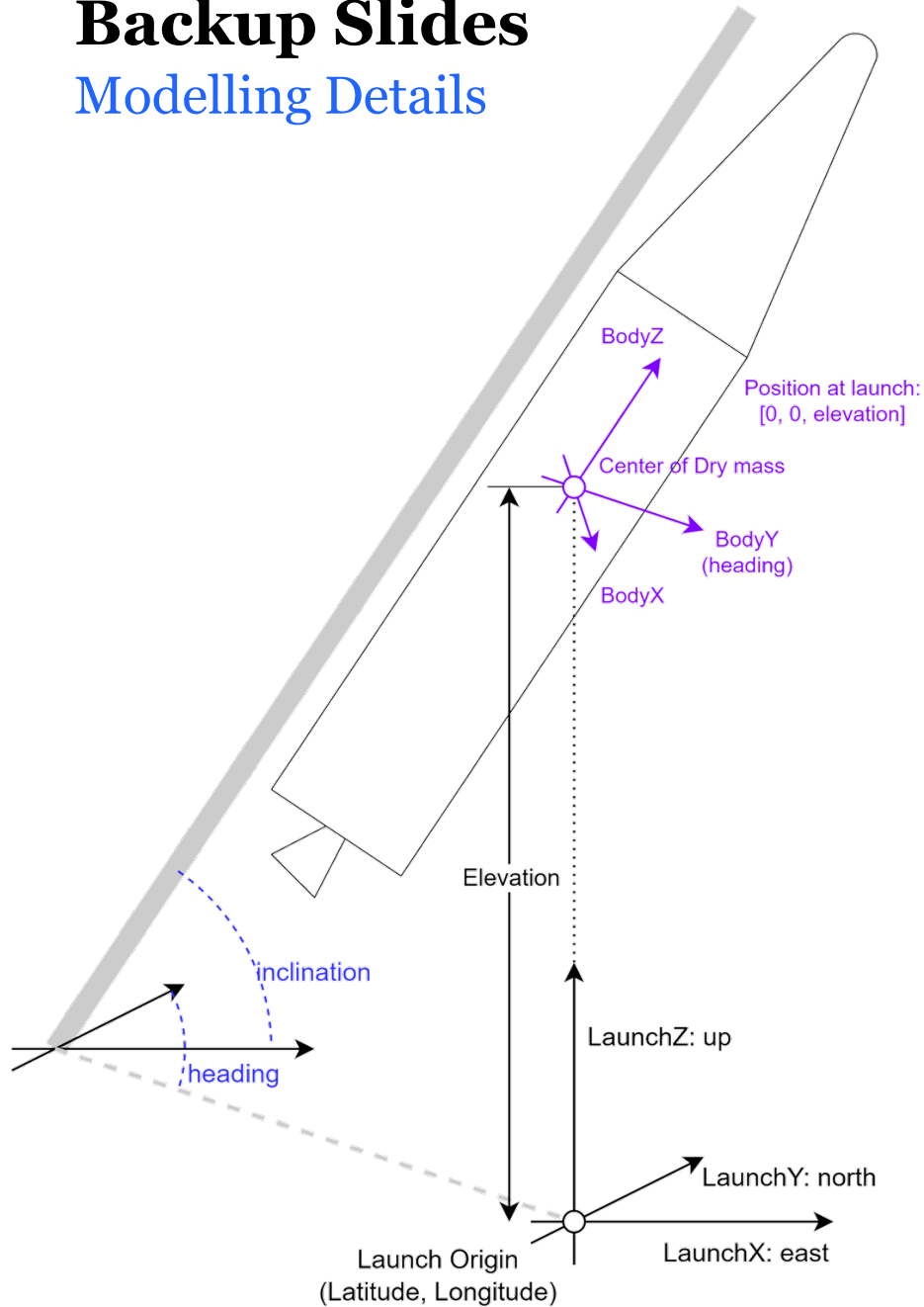
A2: Yes, the RocketPy simulation is effectively paused at step to wait for agent action for simulation of next step. Actuator dynamics (delay, low pass), are not in scenario 0 and 1, but will be introduced in the future.

Second Workshop:

Checkout this [GitHub discussion thread](#).

Backup Slides

Modelling Details



- actions:

- `launch` : a binary command to launch the rocket.
- `launch_inclination_heading` : a 2-element array [inclination, heading] representing the launch inclination (0-90 degrees from horizontal) and heading angles (0-360 degrees from north).
- `tvc` : a 2-element array [TVC_x, TVC_y] representing the thrust vector control (TVC) gimbal angles (deg). Polarity: positive gimbal angles provide positive torques.
- `throttle` : a scalar representing the throttle ratio between 0 and 1.
- `roll` : a scalar representing the roll torque command in N-m.

- observations:

- `simulation_time` : the current simulation time in seconds.
- `balloon_status` : a n-element array representing the status of each balloon (0: on the ground, 1: released, 2: popped). n is the number of balloons in the scenario.
- `balloon_states` : a n x 6 array representing the position (posX, posY, posZ) and velocity (velX, velY, velZ) of each balloon.
 - Position is the center of the balloon in the launch frame (relative to launch origin) in meters.
 - Velocity is the center of the balloon in the launch frame (relative to launch origin) in m/s.
- `rocket_sensors` : a 12-element array representing the rocket's sensor measurements (gyroX, gyroY, gyroZ, accX, accY, accZ, posX, posY, posZ, velX, velY, velZ). Orientation of inertial sensors matches body frame. The measurements will be nan before launch action.
 - Gyroscopes measure the angular velocity (rad/s) in the rocket body frame.
 - Accelerometers measure the linear acceleration (m/s²) in the rocket body frame. Gravity is included in the accelerometer measurements.
 - GNSS sensors measure the position (m) and velocity (m/s) in the launch frame (relative to launch origin).
- Note that the rocket's true states (e.g., attitude, angular velocity) are not directly observed by the agent, and the agent needs to infer them from the sensor measurements.

- info:

- `rocket_states` : a 13-element array representing the rocket's true states. These states are not observed and should not be used by the agent but can be used for development and debugging. The states are [posX, posY, posZ, velX, velY, velZ, e0, e1, e2, e3, wX, wY, wZ]:
 - pos: center of dry mass position (m) in the launch frame (relative to launch origin).
 - vel: center of dry mass velocity (m/s) in the launch frame (relative to launch origin).
 - e: quaternion representing the attitude of the rocket (e0, e1, e2, e3) relative to the launch frame.
 - w: angular velocity (rad/s) in the rocket body frame.

